FINE-TUNING QWEN3-VL-2B FOR ASL TO ENGLISH TRANSLATION*

Ma'moun Yosef
Department of Artificial Intelligence
University of Jordan
Amman, Jordan
mam0233493@ju.edu.jo

May 18, 2026

ABSTRACT

This report describes an end-to-end pipeline for translating continuous American Sign Language (ASL) video into English text by fine-tuning the 2-billion-parameter *Qwen3-VL-2B-Instruct* vision and language model. The training recipe combines a multi-tier LoRA / RSLoRA adaptation with per-tier learning rates and gradient-clipping budgets, a two-corpus staged schedule that pretrains on OpenASL before fine-tuning on How2Sign, a preprocessing pipeline of pose-guided signer cropping, CLAHE contrast enhancement, and pre-extracted MediaPipe landmark overlays, and an InfoNCE auxiliary loss anchoring the pooled vision embedding to its caption embedding. On a custom $90/5/5$ stratified split, the trained model produces fluent English in the register of the target captions and often captures the meaning of the signed input, but the word-level overlap with references is modest (test BLEU-4 $= 1.64$, ROUGE-L $= 10.43$ on the How2Sign test partition). The methodological caveats that bound this result are stated in detail. The report's contribution is a transparent engineering account rather than a state-of-the-art claim.

1 Introduction

Sign languages are full natural languages, expressed through coordinated movement of the hands, face, and upper body. They are a primary means of communication for a substantial fraction of the more than 430 million people worldwide estimated by the World Health Organization to live with disabling hearing loss [1]. Outside the Deaf community, signed messages usually rely on a human interpreter to reach a wider audience. Automatic sign language translation aims to narrow this gap by mapping recorded signing directly to text in a target language. The task addressed here is *continuous* American Sign Language (ASL) to English translation: given a video clip of a signer producing one or more full sentences, the system must produce the corresponding English text.

The task is much harder than spoken-language machine translation. The input is a video stream rather than a sequence of discrete tokens, so timing and meaning have to be inferred together. ASL also carries meaning through several channels at once, including facial expression and body posture, any of which can change the sense of a manual sign. Sign boundaries are not marked explicitly, public corpora are small by the standards of modern language modeling, and pretrained video-capable models are trained mostly on natural imagery, not on the close-range, fast-motion footage typical of signing.

This report describes the third of three attempts at the problem. The first two prototypes used custom vision front ends (MediaPipe landmarks, then a pretrained CNN) feeding an MLP, a Transformer encoder, and a pretrained LLM decoder with all components trainable; neither produced coherent translations. The current attempt replaces that pipeline with a single pretrained model that natively accepts video, *Qwen3-VL-2B-Instruct*, fine-tuned end-to-end with a recipe that fits the available compute. The recipe combines a multi-tier LoRA / RSLoRA adaptation, a two-corpus staged

*Code and full report: github.com/mamounyosef/sign-language-bridge.
Trained weights: huggingface.co/mamounyosef/sign-language-bridge.

schedule (OpenASL then How2Sign), an easy-first curriculum within each stage, and a preprocessing pipeline that crops the signer, enhances contrast with CLAHE, and overlays pre-extracted MediaPipe landmarks on the input frames. No state-of-the-art claim is made; the data splits were built locally rather than taken from the released splits, and the consequences for comparability are discussed in Section 10.

2 Background: Parameter-Efficient Adaptation

Full fine-tuning of a 2-billion-parameter vision and language model is impractical on a single GPU, so the adaptation in this work is restricted to a small set of trainable parameters using Low-Rank Adaptation (LoRA) [2]. LoRA freezes the pretrained weights and inserts a trainable update of the form $\Delta W = BA$ alongside each adapted linear layer, where A and B are low-rank matrices of rank $r \ll \min(d_{\text{in}}, d_{\text{out}})$. Only A and B are updated during training, which reduces the number of trainable parameters by orders of magnitude relative to full fine-tuning while leaving the inference graph unchanged once the update is merged.

At training time the update is scaled by a factor α/r. As shown by Kalajdzievski [3], this scaling causes the effective learning rate to shrink as r grows, which limits the benefit of increasing the rank. Rank-stabilized LoRA (RSLoRA) replaces the factor with $\alpha/\sqrt{r}$, which separates the effective learning rate from r and allows the rank to be tuned per layer without retuning the learning rate. RSLoRA is used throughout this work; the per-layer ranks chosen for the vision encoder, the language model, the output head, and the auxiliary contrastive head are specified in Section 6.

3 Datasets

Two corpora of paired ASL video and English text are used in this work: How2Sign [4] and OpenASL [5]. They differ markedly in domain, size, and caption quality, and are used at different stages of training (Section 6).

3.1 How2Sign

How2Sign [4] is a multi-view ASL corpus of roughly 80 hours of "How To" instructional content with manually verified English transcripts. Its captions are short, well-formed sentences, and the recording conditions are comparatively controlled, with a limited set of signers and consistent framing. In this work How2Sign serves as the higher-quality fine-tuning corpus.

3.2 OpenASL

OpenASL [5] is a roughly 300-hour open-domain corpus collected from online ASL video, with English captions taken from existing subtitle tracks. The captions are noisier than How2Sign's, including occasional ASR-style errors, and the material covers a broader range of register, topic, signer identity, and recording quality. OpenASL serves as the larger but noisier pretraining corpus.

3.3 Duration Filtering and Custom Splits

A duration filter restricts both corpora to clips between 1.3 and 8 seconds. The choice is motivated by the intended deployment scenario: the model is expected to run in near-real-time on short windows of incoming signing, on the order of a few seconds at a time, so training on clips longer than the typical inference window would not reflect the system's actual operating regime. Shorter clips, in turn, frequently contain only fragments of signing. After filtering, a combined pool of $51{,}499$ clips remains, of which $32{,}628$ (63.4%) come from OpenASL and $18{,}871$ (36.6%) from How2Sign.

The official released splits of How2Sign and OpenASL were not preserved. Several cleaning passes were applied to both corpora before splitting: exact-duplicate removal, cross-split data-leakage removal, a words-per-second filter dropping rows with $\text{word\_count}/\text{duration\_sec} > 5.0$ (implausibly fast captions), and fuzzy near-duplicate removal applied both within and across the original splits. In addition, a set of eight light text-normalisation passes was applied to all captions, covering steps such as stripping non-ASCII characters, normalising smart quotes and dashes to ASCII equivalents, collapsing repeated punctuation, sentence-casing all-caps text, and dropping rows with no alphabetic content. The cleaning passes left the original per-split sizes unbalanced across the two sources, so the cleaned data was pooled and a single stratified 90/5/5 random split was applied to the combined pool, stratified by source so that each split preserves the 63.4%/36.6% source ratio. The resulting train, validation, and test sets contain $46{,}349$, $2{,}575$, and $2{,}575$ clips respectively, with 59.74 hours of training video in total. Per-source counts are summarised in Table 1.

Table 1: Final stratified 90/5/5 splits after the 1.3-to-8-second duration filter. Column percentages are within-source; the overall train/val/test split is $90.0\%/5.0\%/5.0\%$ for both sources.

Source	Total	Train	Val	Test
How2Sign	18,871	16,983	944	944
OpenASL	32,628	29,366	1,631	1,631
Combined	51,499	46,349	2,575	2,575

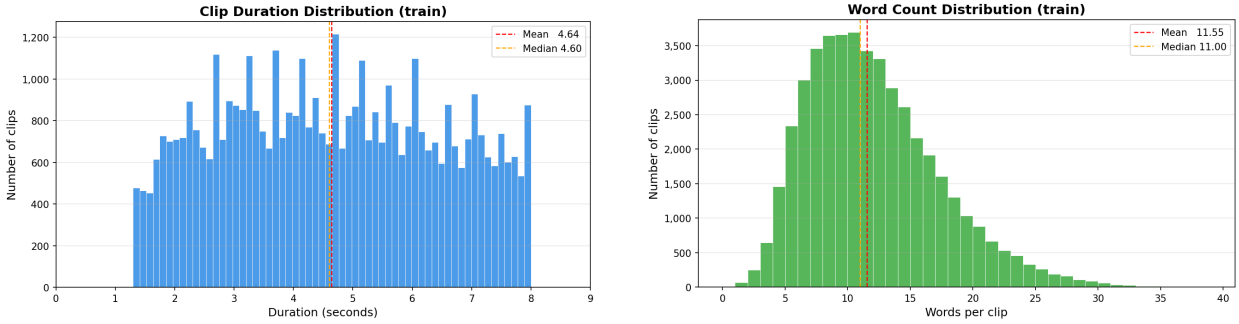

Figure 1: Distributions over the combined cleaned pool. Left: clip duration in seconds, restricted to the 1.3-to-8-second filter window. Right: caption word count.

4 Data Preprocessing Pipeline

Every clip is passed through a fixed sequence of per-frame transforms. Three always-on visual stages are applied in order, illustrated in Figure 2, followed by a stochastic augmentation suite that is active only during training. The frame rate and the per-frame-pair pixel budget are set per source. OpenASL clips are decoded at 18 frames per second with a maximum of $150 \times 32 \times 32 \approx 392 \times 392$ pixels per frame pair, while How2Sign clips are decoded at 20 frames per second with a maximum of $180 \times 32 \times 32 \approx 429 \times 429$ pixels per frame pair. A clip-level cap of $20{,}480 \times 32 \times 32$ pixels across all frames is also enforced.

4.1 Signer Cropping

A single static bounding box per clip is used to crop each frame to the signing region before any further processing. The bounding boxes are computed offline using MediaPipe pose detection and stored alongside the dataset. Cropping before the Qwen3-VL processor's internal resize ensures that the pixel budget lands on the signer rather than on background.

4.2 CLAHE Contrast Enhancement

Contrast-Limited Adaptive Histogram Equalisation (CLAHE) is applied to every cropped frame. The frame is converted to LAB colour space, CLAHE with a clip limit of 2.0 and an 8×8 tile grid is applied to the L channel, and the result is converted back to RGB. Operating on the L channel preserves hue while enhancing local contrast, which makes hand outlines and finger separations easier to read under the varied lighting conditions seen across the two corpora.

4.3 Landmark Overlay

After CLAHE, a skeleton-like overlay of pre-extracted MediaPipe landmarks is drawn on every frame. Each clip carries 21 keypoints per hand and 6 upper-body joints (shoulders, elbows, and wrists), with anatomically connected keypoints joined by thin line segments. The left hand, the right hand, and the upper-body skeleton are drawn in distinct colours so that the vision encoder receives an explicit structural cue alongside the underlying imagery, which is particularly useful on lower-resolution or motion-blurred frames.

4.4 Training-Time Augmentation

A stochastic augmentation suite is applied only at training time, after the three always-on stages. The active augmentations are temporal jitter (small frame shifts with edge replication), affine perturbation (a few degrees of rotation, a

few percent of translation, mild zoom), colour jitter (small brightness, contrast, saturation, and hue swings), random grayscale, and speed perturbation (resampling the frame sequence to simulate slightly slower signing). Augmentation is disabled during the first epoch of each training phase and turned on from the second epoch onward (so that the model can see each original dataset in full at least one time); full parameter values are listed in Appendix B.

Figure 2: Always-on preprocessing stages applied to every clip. From left to right: raw decoded frame; after signer cropping using the precomputed MediaPipe pose bounding box; after CLAHE contrast enhancement on the L channel of LAB; after the MediaPipe landmark overlay (21 keypoints per hand plus 6 upper-body joints).

5 Model Architecture

The model used in this work is the Qwen3-VL-2B-Instruct backbone [6] augmented by two small linear projection heads added on top to support an auxiliary contrastive loss. All other components are inherited from the released checkpoint.

5.1 Qwen3-VL-2B Backbone

The backbone consists of a vision tower coupled to a decoder-only language model via a small connector. The vision tower is a 24-layer Transformer with hidden size 1024, 16 attention heads, and an intermediate MLP size of 4096. Frames enter the tower through a three-dimensional patch embedding (Conv3d) with spatial patch size 16 and temporal patch size 2, so each visual token corresponds to a 16×16 pixel region pooled across two consecutive frames. The tower's 1024-dimensional outputs are projected to the 2048-dimensional language-model embedding space by a Patch Merger module. In addition, three DeepStack mergers extract features from vision-tower layers 5, 11, and 17 and inject them into the language stream alongside the final merger output, giving the language model multi-scale access to the visual input.

The language model is a 28-layer Qwen3-architecture decoder with hidden size 2048, intermediate size 6144, 16 query heads, 8 key-value heads (grouped-query attention), and tied input and output embeddings over a vocabulary of $151{,}936$ tokens. Multimodal Rotary Positional Embeddings (M-RoPE) are used so that the same positional encoding handles text and video tokens consistently. The total backbone parameter count is approximately 2 billion.

5.2 InfoNCE Projection Heads

Two small linear projection heads are added on top of the released checkpoint, mapping the pooled vision and text representations into a shared 256-dimensional embedding space for the InfoNCE contrastive loss described in Section 6. Each head is a single bias-enabled linear layer of shape $2048 \to 256$, initialised with a normal distribution of standard deviation 0.02. The heads are trained from scratch alongside the LoRA adapters, and their weights are saved and restored separately at checkpoint time.

6 Training Methodology

The training recipe combines four components: a multi-tier LoRA adaptation that splits the trainable parameters across the vision tower, the language model, the output head, and the InfoNCE projection heads (Section 6.1); a two-corpus staged schedule that pretrains on OpenASL before fine-tuning on How2Sign (Section 6.2); within-phase tier freezing for OpenASL (Section 6.3); and an easy-first curriculum combined with bucket batching by clip duration (Section 6.4). Loss composition, optimisation, gradient clipping, and checkpointing are covered in Sections 6.5 to 6.8.

6.1 Multi-Tier LoRA Configuration

The trainable parameters are partitioned into four *tiers*, each with its own LoRA rank, scaling factor, learning rate, and gradient-clipping budget. The motivation is that the vision tower, the language model, the output head, and the contrastive projection heads have very different gradient magnitudes, adaptation needs, and risks of catastrophic forgetting, and treating them as a single group forces a compromise that under-serves all of them. RSLoRA (Section 2) is used throughout so that the effective learning rate per tier is decoupled from its rank.

Tiers 1 to 3 are low-rank adapters injected into existing linear and embedding modules of the backbone. Tier 4 is not a LoRA adapter: the two InfoNCE projection heads (Section 5.2) are trained from scratch as full-rank linear layers but are scheduled and clipped under the same tier framework for uniformity. The scaling factor α is set to twice the rank for each LoRA tier ($\alpha = 32, 64, 16$ for tiers $1, 2, 3$ respectively), consistent with the RSLoRA $\alpha/\sqrt{r}$ scaling. Table 2 summarises each tier's configuration and parameter budget; in total $34{,}321{,}920$ parameters are trained, corresponding to 1.59% of the $2{,}161{,}853{,}952$-parameter combined model.

Table 2: Multi-tier trainable-parameter configuration. Ranks apply to tiers 1 to 3 only; tier 4 consists of two full-rank linear projection heads.

Tier	Description	Rank	Layers	Trainable params
T1	LM attention + MLP	16	196	17,432,576
T2	Vision encoder	32	106	14,606,336
T3	Embeddings + output head	8	1	1,231,872
T4	InfoNCE projections (full)	—	3	1,051,136
Total trainable				34,321,920
Frozen base model				2,127,532,032
Total parameters				2,161,853,952

6.2 Two-Corpus Staged Training

Training proceeds in two stages. The first stage runs for two epochs on the larger but noisier OpenASL corpus and is intended to push the vision tower out of its natural-imagery distribution into the signing domain. The second stage runs for six epochs on the cleaner How2Sign corpus and is intended to sharpen the model's output distribution against well-formed target sentences. At the boundary between the two stages, the cosine learning-rate schedules are reset, with the How2Sign-stage peak learning rates set lower than the OpenASL-stage peaks (Section 6.6). The motivation is that once the model has reached a usable state at the end of OpenASL training, the How2Sign stage should refine rather than overwrite the existing weights.

6.3 Within-Phase Tier Freezing for OpenASL

The OpenASL stage is itself split into two phases. During Phase 1, which covers the first 20% of optimiser steps in the OpenASL stage, only the vision-side tiers (T2 and T4) receive gradients; the language-model tiers (T1 and T3) are frozen. The intent is to let the vision tower close the gap to the signing distribution before any updates are made to the language side, reducing the risk that the strong language prior of the pretrained LM absorbs the initially-noisy visual gradient. From Phase 2 onward, all four tiers are active. The How2Sign stage uses a single-phase schedule with all four tiers active from step zero.

6.4 Curriculum and Batch Construction

An easy-first curriculum is applied during the first within-stage epoch of each stage: samples are sorted by clip duration so that shorter clips are presented before longer ones, on the assumption that shorter clips carry fewer co-articulated signs and are therefore easier to learn from. From the second within-stage epoch onward, batches are shuffled. Both stages additionally use bucket batching by clip duration, grouping clips of similar length into the same batch to reduce padding waste in the Qwen3-VL processor. Figure 3 summarises the complete training schedule, including the tier freezing of Section 6.3 together with the curriculum and augmentation toggles introduced above.

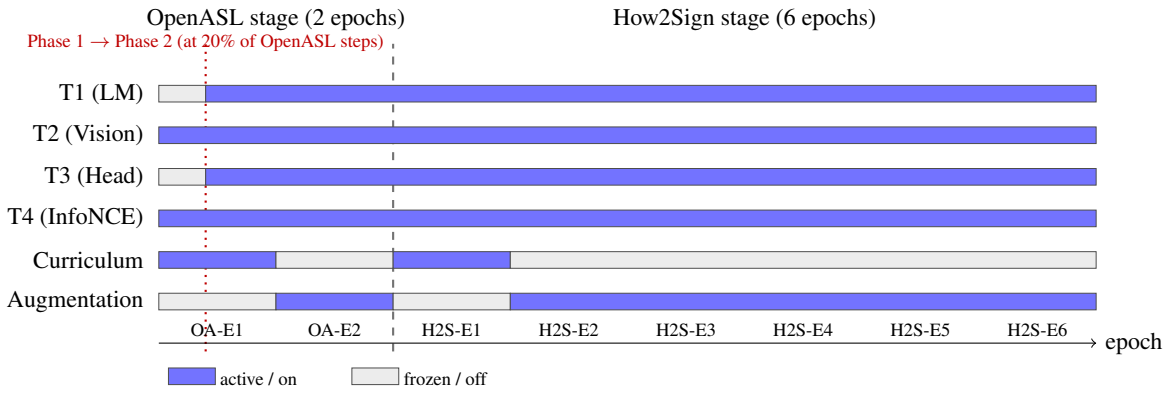

Figure 3: Training schedule over the eight training epochs. For each tier (T1–T4), the bar indicates whether the tier is receiving gradient updates (filled) or frozen (grey). The bottom two rows show whether the easy-first curriculum and the stochastic augmentation suite are active. T2 and T4 are active from the first optimiser step; T1 and T3 wake up at the Phase 1 to Phase 2 boundary inside the OpenASL stage and remain active for the rest of training. The curriculum is applied only during the first within-stage epoch of each stage. Augmentation is the opposite: off during the first within-stage epoch and on from the second epoch onward.

6.5 Loss Composition

The training objective is the sum of two terms. The primary term is the standard next-token cross-entropy loss between the model's predicted distribution and the reference English sentence, computed with label smoothing of 0.04 to soften the target distribution against the noisier OpenASL captions. The auxiliary term is an InfoNCE contrastive loss that encourages the pooled vision embedding of a clip to be closer to the pooled embedding of its matching reference caption than to the embeddings of non-matching captions in the same effective batch. Concretely, let v_i and t_i denote the ℓ_2-normalised projection-head embeddings (Section 5.2) of clip i and its caption, and let $\tau = 0.07$ be the temperature. The video-to-text direction of the InfoNCE loss is

$$\mathcal{L}_{v \to t} = -\frac{1}{N} \sum_{i=1}^{N} \log \frac{\exp(v_i^\top t_i / \tau)}{\sum_j \exp(v_i^\top t_j / \tau)},$$

and the text-to-video direction is defined symmetrically. The negative pool for t_j includes the current batch and a MoCo-style queue of 64 recently-seen detached caption embeddings, giving stronger contrastive pressure than the batch alone provides. The combined loss is $\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \, (\mathcal{L}_{v \to t} + \mathcal{L}_{t \to v})/2$, with $\lambda = 0.3$ throughout, linearly warmed up over the first 200 steps to prevent gradient explosions from the freshly initialised projection heads.

6.6 Optimiser and Per-Tier Learning-Rate Schedules

The optimiser is AdamW with $\beta = (0.9, 0.98)$ and a weight decay of 0.01, using the 8-bit AdamW implementation to reduce optimiser-state memory. Each tier owns an independent peak and floor learning rate per stage, summarised in Table 3. A shared linear warmup ratio of 5% of each tier's active step count is followed by cosine decay to the floor. Tiers 1 and 3 have a smaller active step count than tiers 2 and 4 in the OpenASL stage because their warmup begins at the Phase 1 to Phase 2 boundary rather than at step zero. The How2Sign-stage peak learning rates are set systematically below their OpenASL counterparts so that the fine-tuning stage refines rather than overwrites the OpenASL solution.

6.7 Per-Tier Gradient Clipping

Each tier is clipped independently rather than under a single global norm, so that a high-norm tier (typically T2, the vision encoder) cannot drag down the gradients of well-behaved tiers. The clipping thresholds are 5.0 for T1, 9.0 for T2 (a more generous limit reflecting the naturally larger gradient magnitudes of the vision adapters), 3.0 for T3, and 2.0 for T4. The joint global gradient norm is still computed and logged for diagnostics but is not used to clip.

6.8 Checkpointing, Resume, and Early Stopping

Checkpoints are written every 10 optimiser steps, retaining the eight most recent. Each checkpoint contains the LoRA adapter weights, the InfoNCE projection-head weights, the per-tier optimiser and scheduler states, the InfoNCE

Table 3: Per-tier peak and floor learning rates per training stage. All tiers use a 5% linear warmup followed by cosine decay.

	OpenASL stage		How2Sign stage	
Tier	Peak LR	Floor LR	Peak LR	Floor LR
T1	3×10^{-5}	1.5×10^{-6}	2×10^{-5}	5×10^{-7}
T2	5×10^{-5}	2.5×10^{-6}	4×10^{-5}	2×10^{-6}
T3	2×10^{-5}	1×10^{-6}	1×10^{-5}	5×10^{-7}
T4	5×10^{-5}	2.5×10^{-6}	3×10^{-5}	1.5×10^{-6}

queue contents, and all random-number generator states required for a bit-faithful resume modulo the inherently nondeterministic augmentation and dropout sources. Validation runs every 80 optimiser steps, and early stopping is applied.

7 Engineering and Infrastructure

7.1 Compute Environment and Training Cost

Training was carried out on a single NVIDIA A100 GPU with 80 GB of memory, rented from a cloud GPU provider. The effective batch size was 24, formed from a per-device micro-batch of 6 samples and 4 gradient-accumulation steps. The OpenASL stage ran for $2{,}448$ optimiser steps and the How2Sign stage ran for $3{,}540$ optimiser steps, giving $5{,}988$ optimiser steps in total. The complete training run took 4 days and 18 hours.

7.2 Memory and Throughput Optimisations

Several memory and throughput optimisations were required to fit the model and its training state within 80 GB of GPU memory. Model weights and activations are kept in bfloat16. Attention is computed with FlashAttention 2, which avoids materialising the full attention matrix and reduces both compute and memory cost. Gradient checkpointing trades activation recomputation for further memory savings. The optimiser state is held in 8-bit AdamW, which reduces memory footprint relative to standard 32-bit AdamW. Liger fused Triton kernels are used to reduce memory and accelerate the forward and backward passes. The PyTorch caching allocator is configured with `expandable_segments:True` to limit fragmentation across batches of variable-length video. On the input side, the DataLoader uses eight persistent worker processes with a prefetch factor of six and pinned host memory, so that video decoding and CPU-side preprocessing overlap with GPU compute.

8 Evaluation Protocol

8.1 Metrics

The primary text-similarity metric is corpus-level BLEU-4, computed with the `sacrebleu` reference implementation. Three additional metrics are reported alongside BLEU. ROUGE-L is the longest-common-subsequence-based F-score between predicted and reference sentences. METEOR is the unigram F-score with stemming and synonym bonuses, computed using NLTK. Word Error Rate (WER) is the corpus-level Levenshtein edit distance between predicted and reference word sequences, divided by the total reference word count and expressed as a percentage. A distinct-2 diversity ratio (number of unique bigrams in the model's generations divided by the total number of generated bigrams) is also tracked as a simple check against output collapse onto a small set of fluent but generic phrases. Independently of the generation metrics, the standard next-token cross-entropy loss and its corresponding perplexity are reported on the held-out captions.

8.2 Generation Configuration

Predicted translations are produced by beam search with a beam size of 5, a length penalty of 0.6, a no-repeat-n-gram constraint of $n = 4$, and a repetition penalty of 1.1, and are limited to at most 32 newly generated tokens. The no-repeat-n-gram constraint and the repetition penalty were chosen to suppress the short pathological loops (such as repeated punctuation or repeating stock phrases) observed in early generations.

9 Results

All results reported in this section are from the checkpoint at global step 4,610, selected on the basis of validation loss. Evaluation is performed on the held-out test set for the How2Sign source (944 clips); the OpenASL test partition was excluded from generation evaluation because no comparable reference translations were retained after cleaning.

9.1 Training Dynamics

Figure 4 shows the training and validation cross-entropy loss over the full 5,988 optimiser steps. The OpenASL stage occupies steps 1 to 2,448 and the How2Sign stage occupies steps 2,449 to 5,988. Training loss decreases steadily throughout the OpenASL stage. At the transition to How2Sign, both curves briefly spike before resuming their downward trend, consistent with a cosine schedule reset and a shift in data distribution. By the end of How2Sign training the validation loss is stable at approximately 2.8.

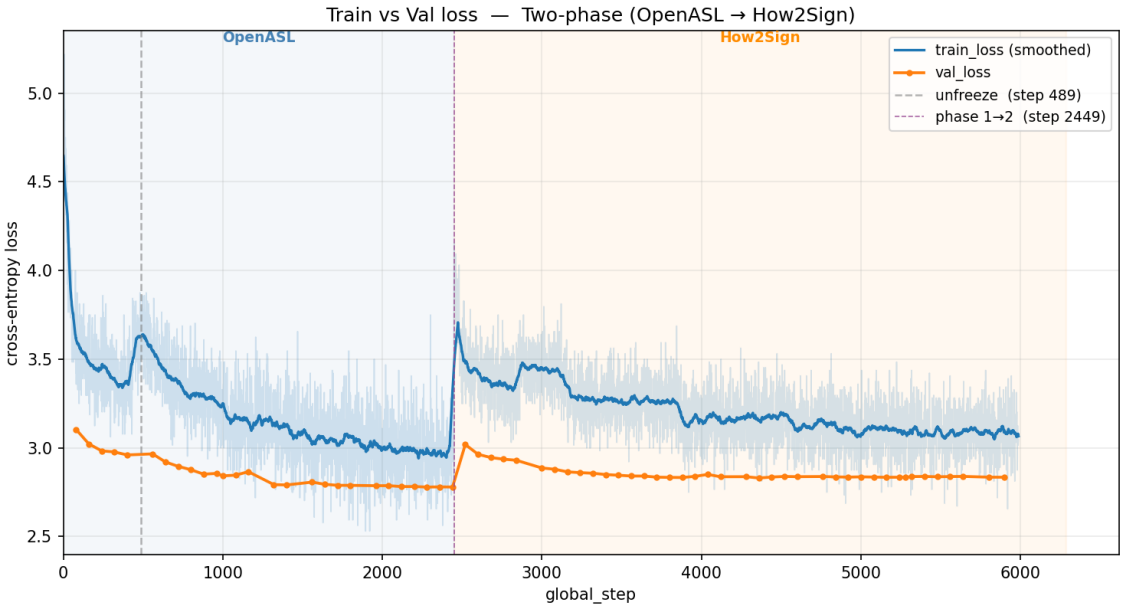

Figure 4: Training and validation cross-entropy loss over the full two-stage training run. The dashed vertical lines mark the Phase 1 to Phase 2 boundary inside the OpenASL stage (step 489) and the OpenASL to How2Sign stage boundary (step 2,449).

9.2 Generation Metrics over Training

Figure 5 shows BLEU-1, BLEU-2, BLEU-4, and ROUGE-L on the How2Sign and OpenASL validation sets, reported at each validation step. Metrics on both sources improve gradually through the OpenASL stage and accelerate visibly once How2Sign fine-tuning begins at step 2,449, at which point the How2Sign curves (blue) rise sharply while the OpenASL curves (orange) plateau. This behaviour confirms that the staged schedule achieves its intended effect: OpenASL pretraining moves the model into the signing domain, while How2Sign fine-tuning sharpens it on the higher-quality target distribution.

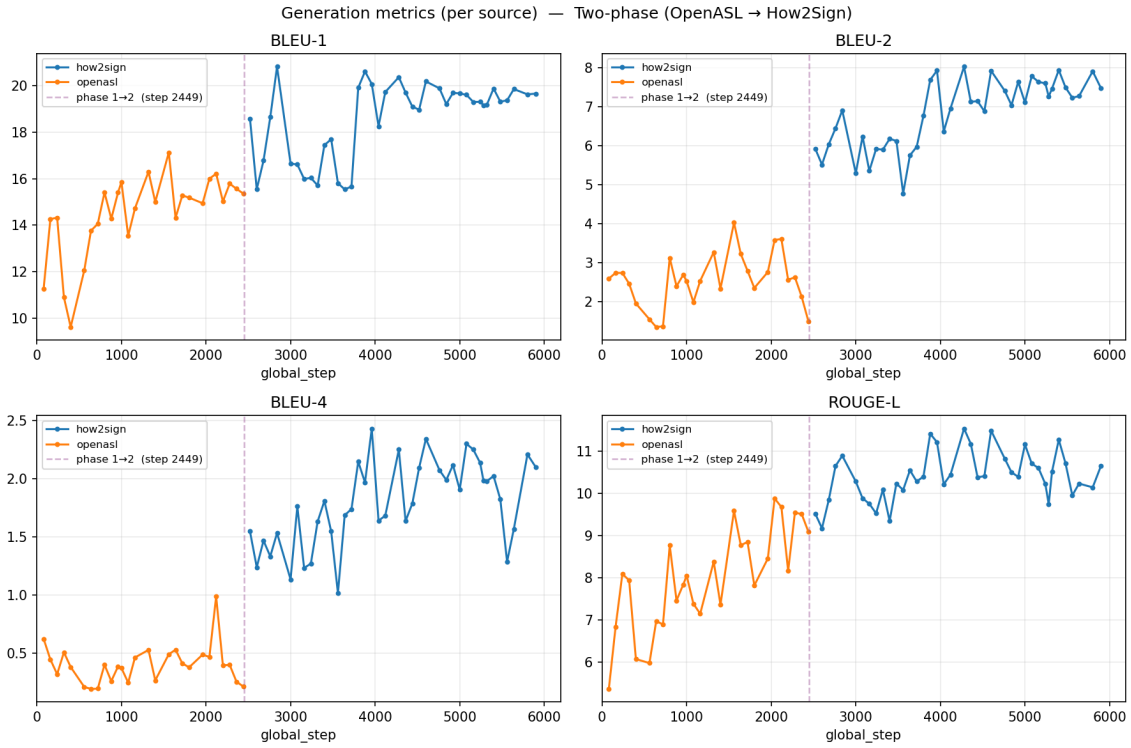

Figure 5: Validation BLEU-1, BLEU-2, BLEU-4, and ROUGE-L over training, plotted per source (How2Sign = blue, OpenASL = orange). The dashed vertical line at step 2,449 marks the OpenASL to How2Sign stage boundary.

9.3 Best-Checkpoint Test Metrics

Table 4 summarises the generation and loss metrics on the How2Sign test set at the selected checkpoint.

Table 4: Test-set results at step 4,610 on the How2Sign test partition (944 clips). Loss and perplexity are computed on teacher-forced reference tokens; all other metrics are computed on beam-search generations.

Metric	Value
Test loss	2.7896
Perplexity	16.28
BLEU-1	19.76
BLEU-2	6.95
BLEU-4	1.64
chrF	17.42
ROUGE-L	10.43
METEOR	9.71
WER (%)	112.51
Distinct-2	0.103
Mean ref. length (words)	12.91
Mean hyp. length (words)	14.39

The model achieves a BLEU-4 of 1.64 and a ROUGE-L of 10.43. These numbers are relatively low, reflecting the small backbone, the limited compute budget, and the inherent difficulty of continuous ASL translation. The word error rate of 112.51% indicates that, while the model produces fluent English, the specific words generated frequently do not match the reference exactly. This behaviour is consistent with a WER above 100%, which arises when the hypothesis is longer

than the reference and the overlapping tokens are few. The distinct-2 score of 0.103 confirms that the model's output is not collapsed onto a small set of repeated phrases, though diversity remains limited.

9.4 Qualitative Examples

Table 5 shows three good generated predictions alongside one clear mismatch.

Table 5: Selected generation examples from the How2Sign test set: three good generated predictions and one representative mismatch.

Reference	Hypothesis
"Well, today I'm going to show you how."	"I'm going to show you how to do that."
"Now with anything, practice makes perfect."	"I'm going to show you how to do that in the next segment."
"It really just takes some practice."	"We're going to do the same thing on the other side."
"Don't panic, don't freak out, it's really not difficult."	"We're going to talk about how to choose a good pair of running shoes."

The best-matching predictions read as natural, fluent English sentences in the same instructional style as the references, and in several cases they capture much of the original meaning, even when the exact wording differs. The mismatch example is a clearer failure: the reference and the hypothesis are about completely different things. Together, these cases show that the model has learned to produce sensible How2Sign-style sentences and can often get the meaning roughly right, but it is not yet reliable at tying a specific clip to its specific caption.

10 Limitations and Lessons Learned

10.1 Custom Data Split

The most consequential methodological caveat in this work is the use of a custom $90/5/5$ stratified split rather than the released How2Sign and OpenASL splits. The choice was driven by the cleaning passes described in Section 3.3, which removed exact duplicates, cross-split near-duplicates, and clips with implausible words-per-second, and left the official per-split sizes unbalanced enough that pooling and re-splitting the cleaned data produced a more uniform evaluation set. The cost is that the numbers reported in Section 9 cannot be accurately compared against published results directly.

10.2 Modest Metrics and High Word Error Rate

As discussed in Section 9.3, the test-set BLEU-4 of 1.64 and word error rate of 112.51% are modest in absolute terms. The qualitative examples in Section 9.4 show that the system is fluent and often captures the meaning of the reference, but the exact wording frequently differs, which the n-gram metrics penalise heavily. There is therefore still room to improve the alignment between generated and reference wording before the system is ready for use in applications that depend on faithful sentence-level translation.

10.3 Single-GPU Compute Budget

All training was performed on a single NVIDIA A100 with 80 GB of memory. This budget shaped several decisions: the choice of a 2-billion-parameter backbone rather than a larger Qwen3-VL variant, the reliance on LoRA rather than full fine-tuning, the use of bfloat16 weights, 8-bit AdamW, and gradient checkpointing, and the cap on per-frame pixel budget. A larger compute budget would have removed or relaxed each of these constraints.

10.4 Lessons Learned

Three short takeaways stand out from the project. First, splitting the trainable parameters into independently-scheduled tiers, rather than applying a single LoRA configuration across the whole model, made the training much easier to reason about and tune. Second, staging a noisier, larger corpus before a cleaner, smaller one produced the clearest inflection in the generation-metric curves at the stage boundary, which suggests the staged schedule was working as intended. Third,

careful data cleaning, in particular the removal of cross-split near-duplicates, mattered more for trustworthy evaluation than any single training-recipe change.

11 Future Work

Two directions stand out as the most promising next steps for this work.

The first is to repeat the same recipe with a larger vision and language backbone. The 2-billion-parameter Qwen3-VL variant used here was chosen to fit within the available single-GPU budget, but the Qwen3-VL family also includes substantially larger variants whose stronger language priors and richer visual representations should help the model produce translations that match the reference more closely at the word level, not just at the level of register and topic.

The second is to expand the training pool with additional clean paired data. The cleaning passes applied in this work removed a noticeable fraction of the original corpora, and the remaining data is dominated by a relatively narrow set of signers and recording conditions. Incorporating further clean ASL-to-English material, whether from larger open corpora or from carefully curated new sources, would broaden the distribution of signers, topics, and recording conditions and is likely to improve generalisation more than any single change to the training recipe.

12 Conclusion

This report has documented an end-to-end pipeline for translating continuous American Sign Language video into English text, built around a fine-tuned Qwen3-VL-2B-Instruct backbone. The contribution is not a new model or a state-of-the-art result but a transparent account of one engineering recipe: a multi-tier LoRA / RSLoRA adaptation, a two-corpus staged training schedule from the larger and noisier OpenASL corpus to the cleaner How2Sign corpus, a preprocessing pipeline combining pose-guided signer cropping, CLAHE contrast enhancement, and pre-extracted MediaPipe landmark overlays, and an InfoNCE auxiliary loss anchoring the pooled vision representation to its caption embedding. The resulting system produces fluent English in the register of the target captions and often captures the meaning of the signed input, while the word-level overlap with reference translations remains modest. The methodological caveats discussed in Section 10, in particular the custom data split, are stated openly so that the numbers in Section 9 are read for what they are: a single, fully documented data point on what is achievable with a small open backbone, a single-GPU budget, and the publicly available ASL-to-English data.

A Full Hyperparameter Configuration

Table 6 consolidates the training, optimisation, loss, and data-handling hyperparameters used in the reported run. Per-tier LoRA ranks are listed in Table 2 and per-tier learning rates are listed in Table 3.

Table 6: Consolidated training-recipe hyperparameters for the reported run.

Hyperparameter	Value
Schedule	
OpenASL stage epochs	2
How2Sign stage epochs	6
OpenASL Phase 1 fraction (T1, T3 frozen)	first 20% of OpenASL steps
Per-stage warmup ratio	5% of tier's active steps
LR decay shape (post-warmup)	cosine to floor
OpenASL optimiser steps	2,448
How2Sign optimiser steps	3,540
Total optimiser steps	5,988
Batching	
Per-device micro-batch	6
Gradient accumulation steps	4
Effective batch size	24
Bucket batching	by clip duration
Curriculum (first within-stage epoch only)	easy-first by clip duration
Optimiser	
Optimiser	8-bit AdamW
(β_1, β_2)	$(0.9,\ 0.98)$
Weight decay	0.01
Per-tier gradient-clipping norm (T1/T2/T3/T4)	$5.0/9.0/3.0/2.0$
Loss	
Label smoothing	0.04
InfoNCE weight λ	0.3
InfoNCE warmup	200 steps (linear)
InfoNCE temperature τ	0.07
InfoNCE MoCo queue size	64
Projection-head dimension	256
Video / vision	
OpenASL fps	18
How2Sign fps	20
OpenASL per-frame-pair pixel budget	$150 \times 32 \times 32 \approx 392{\times}392$
How2Sign per-frame-pair pixel budget	$180 \times 32 \times 32 \approx 429{\times}429$
Per-clip pixel cap	$20{,}480 \times 32 \times 32$
CLAHE clip limit, tile grid	2.0, 8×8 (L channel, LAB)
Validation and checkpointing	
Validation cadence	every 80 optimiser steps
Checkpoint cadence	every 10 optimiser steps
Retained checkpoints	8 most recent
Early stopping	enabled

B Augmentation Parameters

Table 7 lists the configuration of the stochastic augmentation suite referenced in Section 4.4. All augmentations are applied per clip, after the three always-on preprocessing stages, and only when augmentation is enabled (from the second within-stage epoch of each stage onward). The augmentations listed as *disabled* are present in the codebase but were turned off for the reported run.

Table 7: Stochastic augmentation suite. Probabilities are per clip.

Augmentation	Probability	Parameters
Temporal jitter	0.40	shift up to ± 2 frames, edge replication
Colour jitter	0.40	brightness ± 0.1, contrast ± 0.1, saturation ± 0.1, hue ± 0.1
Random grayscale	0.07	—
Affine	0.25	rotation $\pm 2^\circ$, translate $\pm 5\%$, scale 0.95–1.00
Speed perturbation	0.15	resample rate 0.9–1.0$\times$
Gaussian blur	disabled	—
Solarise	disabled	—
Random erasing	disabled	—
Histogram equalise	disabled	(replaced by always-on CLAHE)

References

[1] World Health Organization, "Deafness and hearing loss." `https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss`, 2026. Accessed: 2026-05-13.

[2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

[3] D. Kalajdzievski, "A rank stabilization scaling factor for fine-tuning with LoRA," *arXiv preprint arXiv:2312.03732*, 2023.

[4] A. Duarte, S. Palaskar, L. Ventura, D. Ghadiyaram, K. DeHaan, F. Metze, J. Torres, and X. Giró-i Nieto, "How2Sign: A large-scale multimodal dataset for continuous american sign language," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[5] B. Shi, D. Brentari, G. Shakhnarovich, and K. Livescu, "Open-domain sign language translation learned from online video," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2022.

[6] Qwen Team, "Qwen3-VL technical report," *arXiv preprint arXiv:2511.21631*, 2025.